#Hashlock.

IMF

(Market)

SMART CONTRACT

Security Audit

Performed on Contracts:

IMF.sol

IMFMoneyMarkets.sol

PepeMoneyOracle.sol

sbIMF.sol

Irm69.sol

Platform

ETH

hashlock.com.au

JUNE 2024

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The IMF team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

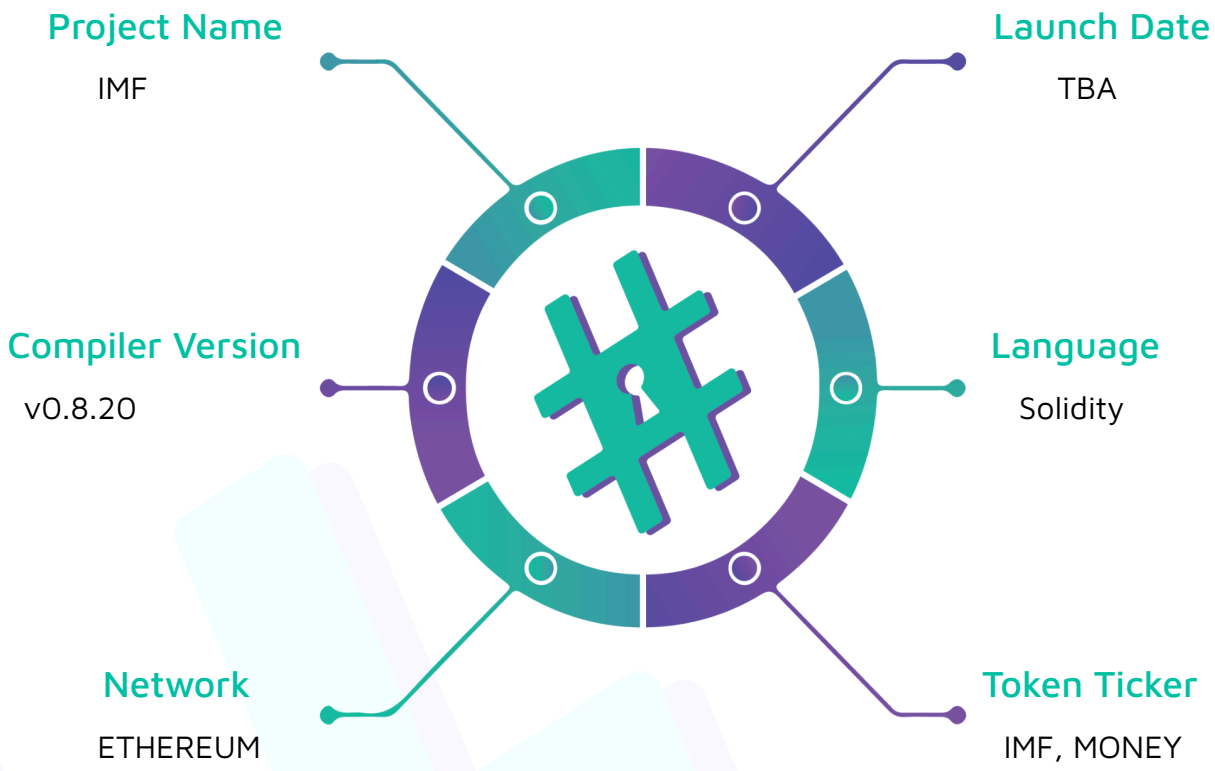
Project Context

IMF is a DeFi dApp built on Ethereum Network that allows users to deposit their tokens and borrow IMF's token $MONEY.

Simply put, users can borrow $MONEY and earn $IMF rewards.

Project Name: IMF
Compiler Version: 0.8.20
Website: internationalmeme.fund
Logo:

Visualised Context:



Project Visuals:

Audit scope

We at Hashlock audited the solidity code within the IMF project, the scope of works included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line by line analysis and were supported by software assisted testing.

Description	IMF Protocol Smart Contracts
Platform	Ethereum / Solidity
Audit Date	June, 2024
Contract 1	IMF.sol
Contract 1 MD5 Hash	c7c91bf8768a0dd34bfd12da376de577
Contract 2	IMFMoneyMarkets.sol
Contract 2 MD5 Hash	0d82d3de635e5f9414f13bff1af914fa
Contract 3	sbIMF.sol
Contract 3 MD5 Hash	404c4c0055a1ed49f72ec5c07c7e4155
Contract 4	Irm69.sol
Contract 4 MD5 Hash	4f7ace012001902c1918c0de33d72827
Contract 5	Money.sol
Contract 5 MD5 Hash	3b5063aaf97eedde0a9a02c35f18d025
Contract 6	PepeMoneyOracle.sol
Contract 6 MD5 Hash	85665869634c2ad84ac1ed363a1427e2
Contract 7	Univ3Oracle.sol
Contract 7 MD5 Hash	391ed151f4016b1e48be1203f58816c2
GitHub Commit Hash	2c13b8104e95877f8eec20d60fd8a239d80b6916

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts. We initially identified some significant vulnerabilities that have since been addressed.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

3 High severity vulnerabilities

1 Medium severity vulnerabilities

2 Low severity vulnerabilities

5 Gas Optimisations

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Behaviours

Claimed Behaviour	Actual Behaviour
IMF.sol - ERC20 token contract	**Contract achieves this functionality.**

IMFMoneyMarkets.sol
- Market contract that allows users to:
  - Deposit collateral to borrow $MONEY
  - Liquidate users with bad debt
  - Calculate health factor, available amount of collateral to borrow with or withdraw
- Allows owner to:
  - Create new markets
	Contract achieves this functionality.

Irm69.sol
- Calculate interest rate on a curve, with the following fixed points:
  - Target price $6.9, rate at target price, 6.9%
  - kink price $4.2, rate at kink 20%
	Contract achieves this functionality.

Money.sol - ERC20 token contract	**Contract achieves this functionality.**
sbIMF.sol - This contract implements the IMF slow burn feature. - Allows users to: - deposit their IMF tokens and receive an NFT, which represent their share in the staking pool.	**Contract achieves this functionality.**

PepeMoneyOracle.sol - PEPE/MONEY oracle contract for price feeds.	**Contract achieves this functionality.**
Univ3Oracle.sol - Uniswap v3 oracle contract for price feeds.	**Contract achieves this functionality.**

Code Quality

This Audit scope involves the smart contracts of the IMF project, as outlined in the Audit Scope section. All contracts, libraries and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however some refactoring is required.

The code is not well commented.

Audit Resources

We were given the IMF projects smart contract code in the form of Github access.

As mentioned above, code parts are not well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

Significance	Description
High	High severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues and inefficiencies

Audit Findings

High

[H-01] PepeMoneyOracle.sol::price - Function returns inflated result

Description

The `poolPrice` function calculates price and multiplies it by `6.9 ether` however due to the implementation of the `price` function this multiplication is done twice.

Vulnerability Details

The `price` function is used to get the price of $MONEY according to $PEPE.

```
function poolPrice(IUniswapV3Pool _pool, bool _dir) public view returns (uint256) {
    (int24 tick, ) = OracleLibrary.consult(address(_pool), period);
    uint160 sqrtRatioX96 = TickMath.getSqrtRatioAtTick(tick);
    uint256 ratioX192 = uint256(sqrtRatioX96) * sqrtRatioX96;
    uint256 usdPrice = _dir
        ? FullMath.mulDiv(ratioX192, 1e18, 1 << 192)
        : FullMath.mulDiv(1 << 192, 1e18, ratioX192);
    return usdPrice * 6.9 ether;
}
function price() external view override returns (uint256) {
    uint256 pepewethPrice = poolPrice(pepeweth, true);
    uint256 wethusdcPrice = poolPrice(wethusdc, false);
    return pepewethPrice * wethusdcPrice / 1e6; // USDC decimals
}
```

This is done via accessing the PEPE/WETH and WETH/USDC pools to get the USDC value of PEPE as shown above.

In the intended use, $PEPE token's USDC price will be multiplied by `6.9 ether` and this will give the price of $MONEY token. However, due to the `price` function making two calls to the `poolPrice` function, this multiplication will be done twice.

Impact

This vulnerability will result in systems relying on the $MONEY price receiving the incorrect value of the token.

Recommendation

Change the functions as shown below:

```
function poolPrice(IUniswapV3Pool _pool, bool _dir) public view returns (uint256) {
    (int24 tick, ) = OracleLibrary.consult(address(_pool), period);
    uint160 sqrtRatioX96 = TickMath.getSqrtRatioAtTick(tick);
    uint256 ratioX192 = uint256(sqrtRatioX96) * sqrtRatioX96;
    uint256 usdPrice = _dir
        ? FullMath.mulDiv(ratioX192, 1e18, 1 << 192)
        : FullMath.mulDiv(1 << 192, 1e18, ratioX192);
    return usdPrice;
}
function price() external view override returns (uint256) {
    uint256 pepewethPrice = poolPrice(pepeweth, true);
    uint256 wethusdcPrice = poolPrice(wethusdc, false);
    return pepewethPrice * wethusdcPrice * 6.9 ether / 1e6; // USDC decimals
}
```

Status

Resolved

[H-02] sbIMF.sol - No locking mechanism results in exploit of stepwise jumps of rewards

Description

The `sbIMF.sol` contract allows users to deposit their IMF tokens and earn rewards. However, lack of a lock mechanism allows this system to be exploited.

Vulnerability Details

Reward tokens will be sent periodically to the `sbIMF.sol` contract. Lack of a lock mechanism in the contract allows any malicious user to sandwich attacking reward emission by depositing their IMF tokens right before and withdrawing them right after. This leads to malicious users keeping their IMF tokens in the contract for only one block and still benefiting from the rewards as they have kept their tokens in the contract for the whole duration.

Proof of Concept

Run the following test in `sbIMFTest.sol`:

```
function testStepwiseJumpExploit() public {

        //mint the tokens for test.

        rewardToken.mint(user2, 5000);

        imfToken.mint(user1, 5000);

        imfToken.mint(user2, 5000);

        imfToken.mint(user3, 5000);



        //user2 deposit tokens

        vm.startPrank(user2);

        imfToken.approve(address(bimf), 5000);

        bimf.mint(1000, 0);
```

```
        //user3 deposit tokens
        vm.startPrank(user3);
        imfToken.approve(address(bimf), 5000);
        bimf.mint(1000, 0);

        //user1 deposit just before reward emission
        vm.startPrank(user1);
        imfToken.approve(address(bimf), 5000);
        bimf.mint(5000, 0);
        //reward emission
        vm.startPrank(user2);
        rewardToken.transfer(address(bimf), 1000);
        //withdraw right after reward emission
        vm.startPrank(user1);
        bimf.withdraw(2, 0, 5000);
        vm.stopPrank();

        console.log(rewardToken.balanceOf(user1));
        console.log(imfToken.balanceOf(user1));
    }
```

Test will print out the following logs:

```
Logs:
  714
  5000
```

This test proves that user1 had their tokens in the contract for 1 block and benefited fully from the reward emission.

Impact

Malicious users will fully benefit from rewards even if they have their tokens deposited for only 1 block.

Recommendation

Implement a locking mechanism for users that deposit their tokens.

Note:
After careful discussion internally with the IMF Team, it was concluded that the finding won't be an issue in practice, as rewards are randomly streamed based on borrow/lend actions as opposed to over some predictable epoch. Regardless, the IMF Team put in a time lock mechanic in the interest of defense in depth.

Status

Resolved

[H-07] PepeMoneyOracle.sol, Univ3Oracle.sol::price - TWAP period set in deployment leaves users vulnerable to price manipulation

Description

The `price` function in the oracle contracts calculates prices from Uniswap pools using TWAP (time weighted average price). TWAP is implemented to protect the Uniswap oracle against price manipulation attacks. It is important to set the TWAP period correctly.

Vulnerability Details

The deployment script sets the `period` for TWAP to 60 seconds as shown below:

```
contract DeployPepeMoneyOracle is Script {

    function run() external {

        vm.startBroadcast();

        new PepeMoneyOracle(60);
```

```
        vm.stopBroadcast();

    }

}
```

Referring to the Uniswap v3 docs most protocols set the TWAP period to 30 minutes to protect their oracle against price manipulation.

Impact

This vulnerability will leave the protocol and its users vulnerable to price manipulation attacks.

Recommendation

Set the TWAP period to 30 minutes in order to follow the most common practices.

Note:
The correct value was internally discussed between the IMF team and the auditors at Hashlock, resulting in the IMF team communicating that it was a placeholder and was decided as a team.

Status

The value provided to us during Audit was a placeholder. The IMF Team requested our advice on the right TWAP duration, which was subsequently implemented.

Medium

[M-01] sbIMF.sol::deposit - Users can deposit IMF tokens into wrong id

Description

In the `deposit` function, no checks are done if the tokenId belongs to the depositing user. This can lead to users depositing into wrong id.

Vulnerability Details

The `sbIMF` contract mints a new NFT for every user's first deposit into the contract. The `deposit` function allows users to deposit tokens into their minted NFT token. However, the `deposit` function has no checks done to see if the id belongs to the user.

```
function deposit(
     uint256 tokenId,
     uint256 assets,
     uint256 shares
 ) public returns (uint256, uint256) {
    require(UtilsLib.exactlyOneZero(assets, shares), ErrorsLib.INCONSISTENT_INPUT);
     burnImf();
     _updateRewardsPerShare();
     if (totalShares == 0) {
         if (assets > 0) shares = assets;
         else assets = shares;
     } else {
         if (assets > 0) shares = assets.mulDivUp(totalShares, totalAssets);
         else assets = shares.mulDivDown(totalAssets, totalShares);
     }
     sharesOf[tokenId] += shares;
     totalShares += shares;
     totalAssets += assets;
     rewardDebt[tokenId] += accRewardPerShare.mulDivUp(shares, 1e12);
     IERC20(imfToken).transferFrom(msg.sender, address(this), assets);
     emit Deposit(msg.sender, tokenId, assets, shares);
     return (assets, shares);
```

```
    }
```

Impact

Having no protection against user errors can lead to users depositing their tokens into the wrong id and losing access to their tokens with no way of receiving them back.

Recommendation

Implement the following access control check in the `deposit` function.

```
function deposit(
        uint256 tokenId,
        uint256 assets,
        uint256 shares
    ) public returns (uint256, uint256) {
        require(UtilsLib.exactlyOneZero(assets, shares), ErrorsLib.INCONSISTENT_INPUT);
        require(ownerOf(tokenId) == msg.sender, "Not token owner");
        burnImf();
        _updateRewardsPerShare();
        //rest of the function...
```

Status

Resolved

Low

[L-01] Contracts - Missing checks for address(0) in constructor/initializers

Description

Missing checks for address(0) in constructor/initializers, it is important to implement address(0) checks to avoid potential mistakes.

There are 4 instances of this issue in the following contracts: `sbIMF.sol`, `Univ3Oracle.sol`.

Recommendation

It is strongly recommended to implement checks to prevent the zero address from being set during the initialization of contracts. This can be achieved by adding require statements that ensure address parameters are not the zero address.

Status

Resolved

[L-02] Contracts - Use Ownable2Step rather than Ownable

Description

Ownable2Step and Ownable2StepUpgradeable prevent the contract ownership from mistakenly being transferred to an address that cannot handle it (e.g. due to a typo in the address), by requiring that the recipient of the owner permissions actively accept via a contract call of its own.

Recommendation

Consider using Ownable2Step or Ownable2StepUpgradeable from OpenZeppelin Contracts to enhance the security of your contract ownership management.

Status

Resolved

Gas

[G-01] sbIMF.sol::merge - Cache Local Variable Array Length In For Loop

Description

If not cached, the solidity compiler will always read the length of the array during each iteration. If it is a memory array, this is an extra mload operation (3 additional gas for each iteration except for the first).

Recommendation

To optimise gas consumption, cache the length of memory arrays before for loop iterations in Solidity.

Status

Resolved

[G-02] sbIMF.sol::merge - State Variable Access Within a Loop

Description

State variable reads and writes are more expensive than local variable reads and writes. Therefore, it is recommended to replace state variable reads and writes within loops with a local variable. Gas savings should be multiplied by the average loop length.

Recommendation

Optimise gas usage in Solidity by replacing state variable reads and writes within loops with local variable operations. This reduces gas costs significantly, especially when multiplied by the average loop length.

Status

Acknowledged

[G-03] IMFMoneyMarkets.sol - address(this) should be cached

Description

Caching saves gas when compared to repeating the calculation at each point it is used in the contract.

Recommendation

To enhance gas efficiency, cache the contract's address by storing address(this) in a state variable at the point of contract deployment or initialization. Use this cached address throughout the contract instead of repeatedly calling address(this).

Status

Acknowledged

[G-04] sbIMF.sol::merge - Revert String Size Optimization

Description

Shortening the revert strings to fit within 32 bytes will decrease deployment time and decrease runtime Gas when the revert condition is met.

Revert strings that are longer than 32 bytes require at least one additional mstore, along with additional overhead to calculate memory offset, etc.

Recommendation

To optimize Gas usage in your Solidity contract, it is recommended to keep revert strings as short as possible and to ensure that they fit within 32 bytes.

Status

Resolved

[G-05] sbIMF.sol::merge - Increments can be unchecked in for-loops

Description

Newer versions of the Solidity compiler will check for integer overflows and underflows automatically. This provides safety but increases gas costs. When an unsigned integer is guaranteed to never overflow, the unchecked feature of Solidity can be used to save gas costs.

Recommendation

Use unchecked math to block overflow/underflow check to save Gas.

Status

Resolved

Centralisation

The IMF project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlocks analysis, the IMF project seems to have a sound and well-tested code base, now that our findings have been resolved/acknowledged in order to achieve security. Overall, most of the code is correctly ordered and follows industry best practices. The code is not well commented. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits are to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and whitebox penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contracts details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment and functionality (performing the intended functions).

Due to the fact that the total number of test cases are unlimited, the audit makes no statements or warranties on security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bugfree status or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds, and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3 oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au